0_IA_2_DE&MLOps

Module 3

1. Explain **ML model with a neat diagram of model development**.
2. Explain **data exploration and feature engineering and selection**.
3. Explain **how evaluation and comparing models is done** and explain how **choosing evaluation metrics perform**. (sic: likely "is performed")
4. Explain **version management and reproducibility in detail**.

Module 4

1. With a neat diagram, explain **preparing for production for ML project lifecycle** along with **runtime environment**.
2. Explain with giving consideration for **adaptation from development to production environment**.
3. Explain **CI/CD pipelines in detail**.
4. Explain **deployment strategies and categories of model deployment**.

Module 5

1. With neat diagram, explain **monitoring and feedback loop highlighted in the larger context of ML lifecycle**.
2. Explain the **feedback loop** along with **logging and model evaluation**.

Module 3

1. Explain **ML model with a neat diagram of model development**.
   Answer:
   ML MODEL WITH A NEAT DIAGRAM OF MODEL DEVELOPMENT

   An ML model is a system that learns patterns from data and makes predictions without being explicitly programmed.

   Meaning of an ML Model

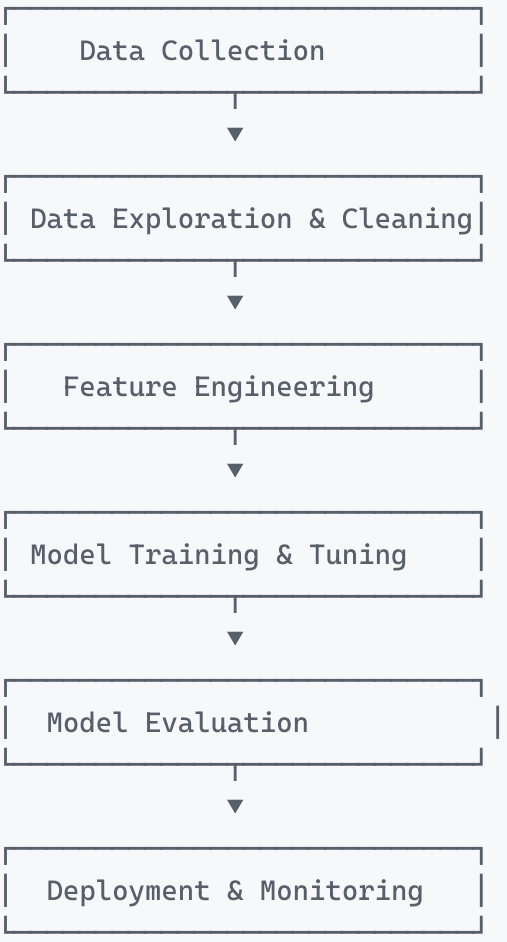
   - An ML model is a **mathematical representation** of a real-world process built from data.
   - It learns from **historical examples** and predicts outcomes for new inputs.
   - It generalizes patterns using algorithms such as regression, decision trees or neural networks.

- In practice, a model consists of **learned parameters** + **data preprocessing steps** used to transform inputs.
- A good ML model should have strong **generalization ability**, meaning it performs well on unseen data.

Components Required to Build an ML Model

- **Training Data** – labeled data used to teach the model patterns.
- **Performance Metric** – defines the goal (accuracy, RMSE, error rate, etc.).
- **ML Algorithm** – method used to learn from data (e.g., logistic regression, SVM, deep learning).
- **Hyperparameters** – settings like learning rate, depth, number of iterations that control learning behaviour.
- **Evaluation Dataset** – unseen data used to test the model's generalization ability.

ML MODEL DEVELOPMENT PROCESS (NEAT DIAGRAM + EXPLANATION)



Explanation of Model Development Steps

- **Data Exploration & Cleaning**
  - Understand data structure, remove missing/outlier values, ensure data quality.
- **Feature Engineering**
  - Convert raw data into meaningful numerical features.
  - Select important features that improve accuracy.
- **Model Training & Experimentation**
  - Train the model using different algorithms.
  - Tune hyperparameters to balance bias–variance.
  - Compare multiple models for best performance.
- **Model Evaluation**
  - Test on unseen data using accuracy, error rate, precision, recall etc.
  - Ensure reproducibility and fairness.
  - Select the best model version.
- **Deployment & Monitoring**
  - Deploy the model into the real-world application.
  - Continuously monitor performance for model drift.
  - Retrain when performance decreases.

---

2. Explain **data exploration and feature engineering and selection**.
   Answer:
   DATA EXPLORATION AND FEATURE ENGINEERING & SELECTION

   1. Data Exploration (Exploratory Data Analysis – EDA)
   This is the first stage of ML model development where the goal is to understand the data before training any model.

   - Helps understand **patterns, structure, errors, and relationships** in data.
   - Ensures that the dataset is **reliable, clean and suitable** for modelling.
   - Supports **hypothesis building**, identifies **cleaning needs**, and guides **feature selection**.

   Activities in Data Exploration

   - **Documentation:** Understand how data was collected and note assumptions.
   - **Statistical Summary:** Check mean, range, missing values, inconsistencies and outliers.
   - **Data Cleaning:** Handle missing values, remove noise, reshape or filter data.

- **Distribution Analysis:** Study how values are spread across each feature.
- **Correlation Analysis:** Identify relationships between variables and the target.
- **Subpopulation Tests:** Compare behaviour across groups to detect biases.
- **Domain Knowledge Use:** Experts help detect unusual data patterns.
- **Governance Checks:** Ensure data privacy, PII removal, and representation of minority groups.

---

2. Feature Engineering and Selection

Features are the **numerical inputs** given to ML algorithms. Feature engineering transforms raw data into meaningful numerical features. This is often the **most time-consuming part** of ML development.

Feature Engineering Techniques

- **Derivatives:** Create new information from existing data.
  - Example: Extract *day of the week* from a date.
- **Enrichment:** Add external data sources.
  - Example: Mark whether a date is a holiday.
- **Encoding:** Convert information into a better machine-readable format.
  - Example: Weekend vs weekday indicator.
- **Combination:** Merge multiple features to form more informative ones.
  - Example: Weighting backlog size by task complexity.

Other Techniques

- **Impact Coding:** Replace a category with its average target value.
- **One-Hot Encoding:** Convert categorical values into binary columns.
- **Embeddings:** Deep learning technique to convert text/images into numerical vectors.

Feature Selection

- Too many features can increase cost, reduce stability and raise privacy issues.
- Too few features may reduce accuracy or fairness.
- Feature selection aims to keep **only the most important features**.

Benefits and Downsides

- More features → can improve accuracy and fairness.

- But → increases computation, maintenance, and risk of over-fitting.

Automated Feature Selection

- Uses heuristics like **correlation with target** to pick useful features.
- AutoML tools can suggest best combinations of features.
- Human judgement is still required for important business or ethical decisions.

Feature Stores

- Centralized repositories for storing and reusing engineered features.
- Improve consistency across offline and online ML systems.

---

3. Explain **how evaluation and comparing models is done** and explain how **choosing evaluation metrics perform**. (sic: likely "is performed")
   Answer:
   EVALUATION AND COMPARISON OF MODELS & CHOOSING EVALUATION METRICS

   1. How Evaluation and Comparison of ML Models Is Done

   Model evaluation checks how well a trained model performs on unseen data, and comparison identifies the best model among alternatives.

   Quantitative Evaluation (Performance-based)

   - Uses numerical metrics such as **accuracy, error rate, precision, recall, RMSE** etc.
   - Evaluation is done on **holdout data** not used during training to check **generalization**.
   - **Cross-testing** evaluates models on the most recent or realistic subset of data to detect data drift.
   - **k-Fold Cross-Validation** splits data into k parts, trains k times, and averages performance for stability.
   - New model versions must be compared with the currently deployed model on **the same evaluation dataset**.

   Qualitative Evaluation (Behavior-based)

   - Checks how the model behaves for different inputs beyond just raw accuracy.
   - Use **input sensitivity analysis** to detect unusual prediction patterns.
   - **Subpopulation analysis** ensures fairness (e.g., similar performance across gender or age groups).

- Models in regulated industries require **explainability** (Responsible AI).
  - Techniques include **Shapley values** and **Individual Conditional Expectation (ICE)** plots.
- Helps detect overly complex “black-box” models that are hard to justify in business contexts.

Model Comparison Tools

- Experiment tracking systems store **datasets, parameters, hyperparameters, features and metrics**, enabling side-by-side comparison.
- Ensures models are reproducible and can be audited later.

---

2. Choosing Evaluation Metrics and How They Perform

The performance metric defines what the model optimizes. Choosing the right metric is crucial because a poor choice leads to **misleading conclusions** and **unintended outcomes**.

Importance of Choosing the Right Metric

- No universal metric fits all tasks; it depends on business goals and data characteristics.
- Wrong metrics cause the **“cobra effect”**, where optimizing the metric worsens real performance.
- Example: Using **accuracy** in imbalanced datasets can be misleading.
  - A model predicting all negatives may get 95% accuracy even if it fails to detect important positive cases.

Key Issues in Evaluation Metric Selection

- **Context-Agnostic Metrics Are Dangerous:** Metrics must reflect business objectives, not only math.
- **Accuracy Is Rarely Enough:** Especially poor for rare-event predictions.
- **Suspiciously Perfect Scores Indicate Problems:**
  - **Data leakage:** Target value appears directly or indirectly in the input features.
  - **Overfitting:** Model memorizes noise instead of learning patterns.

Goal of Evaluation

- The aim is not a perfect metric score but a model that is **reliable, stable, and useful** in real-world deployment.

- Even modest improvements can yield significant business value (e.g., small improvement in demand forecasting can save large costs).

---

4. Explain **version management and reproducibility in detail**.
   Answer:
   VERSION MANAGEMENT AND REPRODUCIBILITY IN MACHINE LEARNING

   1. Version Management

   Version management is essential because ML model development is highly iterative. Data scientists try multiple algorithms, features, hyperparameters and may need to revert to earlier experiments.

   - Ensures the ability to **return to any previous experiment**, configuration, or model state.
   - Helps track **changes in code, data pipelines, preprocessing steps**, and model parameters.
   - Prevents loss of progress when experimenting with multiple ideas.
   - Makes it possible to compare different experiment branches and restore older versions if new ones fail.
   - Traditional **source control systems** (Git) handle code versioning.
   - Modern ML platforms expand versioning to **data transformations, feature pipelines, experiment logs, and model files**.

   ---

   2. Reproducibility

   Reproducibility is the ability to **re-run the exact same experiment** and obtain the same results, even months or years later. It is crucial for auditing, debugging, and deploying reliable models.

   - Needed for scientific validity, model governance, and regulatory compliance.
   - Ensures that model results in production match what was tested during development.
   - Allows new team members to understand and continue previous work without ambiguity.
   - Helps detect issues like environment drift, data drift, and hidden assumptions.

Key Facets Required for Reproducibility

To reproduce a model exactly, several components must be tracked, versioned, and re-usable:

- **Assumptions:**
  - All problem definitions, data scope decisions, and modeling assumptions must be explicitly logged.
  - Helps future teams understand why certain choices were made.
- **Data:**
  - The **same training dataset** must be available.
  - Versioning data is challenging because data is large and frequently updated.
  - Snapshotting or hashing datasets is often required.
- **Settings:**
  - All preprocessing steps, feature transformations, and hyperparameters must be preserved.
  - Any change in settings can change the model output.
- **Randomness Control:**
  - Many algorithms rely on pseudo-random numbers (sampling, initialization).
  - Reproducibility requires fixing a **random seed** to obtain identical results.
- **Results Documentation:**
  - Store confusion matrices, error curves, feature importance, partial dependence plots, etc.
  - Allows comparisons between versions for auditing.
- **Implementation Consistency:**
  - Different implementations (batch vs. API scoring) may produce slightly different outputs.
  - Ensuring consistent logic across environments is essential.
- **Environment:**
  - ML models are sensitive to software versions (Python, libraries, OS).
  - Production must match development versions to prevent scoring differences.
  - **Containerization** (e.g., Docker) helps freeze the computation environment.

Automation and Tools

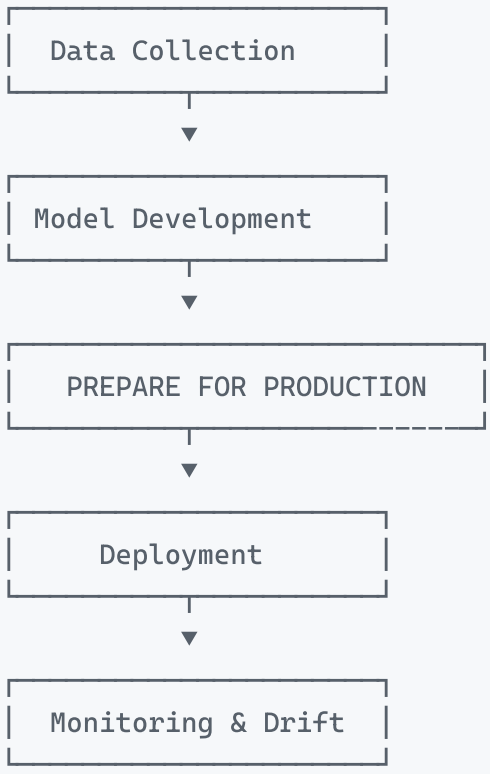
- Integrated MLOps platforms automate much of version tracking and documentation.
- This reduces human error and ensures smooth transition from model development to deployment.

- Automated reproducibility improves long-term stability, debugging, and trust in deployed systems.

Module 4

1. With a neat diagram, explain **preparing for production for ML project lifecycle** along with **runtime environment**.
   Answer:
   PREPARING FOR PRODUCTION IN THE ML PROJECT LIFECYCLE (WITH DIAGRAM)
   Preparing for Production is the phase where the validated model is checked for **technical readiness, reliability, stability, and safety** before deployment. The goal is to ensure the model behaves correctly in real-world environments, not just in development.

Neat Diagram – Position of "Prepare for Production"



---

1. Meaning of Preparing for Production

- Ensures the model that worked in development will also work **safely and efficiently in real-world production systems**.
- Reduces **commercial risk** and avoids unexpected failures after deployment.
- Produces two key outputs:
  - A **production-ready model**
  - **Verification tests and monitoring setup**

Key areas include:

- Runtime environment preparation
- Risk evaluation
- Quality assurance
- Reproducibility
- Security checks
- Mitigating deployment risks

---

2. Runtime Environment (Core Component of Production Readiness)
The runtime environment is the **actual system** where the model will run after deployment. Evaluating and preparing the runtime ensures that the model behaves the same in production as it did during development.

A. Heterogeneity and Adaptation

Production environments may differ from the development environment, such as:

- Cloud services, on-prem servers
- TensorFlow Serving, Kubernetes, REST APIs
- Embedded systems, JVM-based systems

This leads to two scenarios:

Minimal Adaptation

- Happens when dev and prod platforms are similar.
- Only a few steps are needed to push the model to production.
- Ensures high consistency of results.

Maximum Adaptation (Reimplementation)

- Model may need to be rewritten in another language (e.g., Python → Java/C++).
- Time-consuming and expensive.
- Requires extensive validation.

Key Principle: Validation must occur in an environment that **mimics production**.

B. Tooling, Format and Performance Considerations

- Production environment may require a **specific model format** (e.g., PMML, ONNX).
- Conversion must be planned early to avoid last-minute blockers.
- For high-traffic applications, inference time performance must be optimized.

Examples:

- Python → C++ conversion for faster inference.
- Reducing model complexity to lower computational cost.

Inference Scales the Most:
A model trained once might be used **millions of times**, so runtime efficiency is crucial.

C. Model Optimization for Large Neural Networks

When models are too large for real-time production, optimization may be needed:

- **Quantization** – Lower precision numbers used during inference (faster, smaller).
- **Pruning** – Removing unnecessary weights/layers to reduce size.
- **Distillation** – Training a smaller model to mimic a larger one.

These are often integrated into **training strategy** to avoid loss of accuracy.

D. Data Access Requirements

A production model must reliably access all required data sources.

The environment must support:

1. **Network access** to databases/external systems
2. **Required libraries/drivers** to connect to data sources

3. **Secure credentials** for authentication

Since data connectivity is a common failure point, **pre-deployment validation** is critical.

Summary (Exam-Friendly)

- Preparing for Production ensures the model is safe, stable and efficient before real deployment.
- It handles runtime compatibility, environment consistency, performance, risk checks and reproducibility.
- Runtime environment preparation ensures the model runs in production exactly as tested in development.
- Includes model format compatibility, data access setup, optimization, and environment duplication.

---

2. Explain with giving consideration for **adaptation from development to production environment**.
   Answer:
   ADAPTATION FROM DEVELOPMENT TO PRODUCTION ENVIRONMENT IN ML

   Adapting a Machine Learning model from the **development environment ("laboratory")** to the **production environment ("real world")** is one of the most challenging steps in the ML project life cycle. The purpose is to ensure that the model behaves reliably and efficiently under real-world constraints.

   Why Adaptation Is Needed

   - Development and production systems are usually **very different**.
   - Production has **higher risk**, tighter performance needs, and strict security requirements.
   - A model working well in development may fail in production if the environment is not compatible.

   ---

   1. The Ideal vs. Real-World Scenario

   - Ideally, the model should be sent to production **as-is** with no changes.
   - This ensures **consistent behaviour** between development and production.

- In reality, this is rarely possible due to differences in platforms, languages, formats, and performance requirements.

---

2. Levels of Adaptation Required

Minimal Adaptation

- Occurs when development and production systems are **interoperable**.
- Example: Same vendor platform on both sides.
- Model can be deployed with a few commands.
- Most effort shifts to **validation and testing**.

Intermediate Adaptation

- Requires **transformations** to make the model productionCompatible.
- Includes converting formats, adjusting code, or modifying data flow.
- Must validate the model in an environment that **mimics production**, not the dev environment.

Maximum Adaptation (Reimplementation)

- Model must be **rewritten from scratch** in another language or framework.
- Example: Python model must run inside a Java/C++ environment.
- Very expensive, slower release cycle, may take months or years.
- Usually results from poor upfront planning or incompatible infrastructure.

---

3. Technical Factors Driving Adaptation

A. Tooling and Model Format Requirements

- Production may require a **specific model format** such as PMML, ONNX, or a Java/C++-compatible object.
- Example: A scikit-learn model must be converted before running in a JVM environment.
- Conversion tooling must be set up **early in development**, not after building the model.
- Without format planning, deployment may get blocked entirely.

B. Performance and Latency Constraints

- Production requires **fast, scalable inference**.
- A Python model may be too slow for real-time scoring → convert to C++ for speed.
- Complex models incur high inference costs, since they may be used **millions or billions of times**.
- Simplifying or compressing the model can significantly reduce production cost and response time.

C. Optimization for Deep Neural Networks

Large models often require special optimization to fit production constraints, especially on low-power devices.

- **Quantization:** Lower precision during inference → faster and smaller.
- **Pruning:** Remove unnecessary neurons or layers → reduces size.
- **Distillation:** Train a smaller model to imitate a large one.

These techniques should influence how the model is trained, not just post-processing steps.

D. Data Access and Connectivity

A production model must have reliable access to all required data sources.

The environment must include:

1. **Network access** to needed databases.
2. **Drivers/libraries** for reading storage systems.
3. **Secure authentication credentials**.

Data access issues are a common cause of production failures, making **production-like validation essential**.

---

4. Importance of Adaptation Planning from the Beginning

- Production constraints like environment, security, latency, and data access must be considered **early in development**.
- Avoids last-minute rework, reimplementation, or deployment failure.
- Ensures smoother transition and reliable model performance after deployment.

---

3. Explain **CI/CD pipelines in detail**.
   Answer:
   CI/CD PIPELINES IN MACHINE LEARNING (DETAILED EXPLANATION)

   CI/CD pipelines are a core part of **MLOps** and belong to the **Deploy to Production** phase of the ML project life cycle. CI/CD ensures ML models are integrated, validated, packaged, and deployed in a **reliable, automated, and repeatable manner**.

Meaning of CI/CD

- **CI – Continuous Integration**
  - Developers frequently merge changes to a **central repository** (e.g., Git).
  - Automated tests run immediately to detect errors early.
  - Prevents integration conflicts and ensures stable, consistent workflow across multiple contributors.
- **CD – Continuous Delivery**
  - After integration, the system **builds and validates a deployable model artifact**.
  - Ensures that every version is production-ready and tested before release.
- **Continuous Deployment (optional extension)**
  - Fully automates the deployment to production without manual approval.
  - This is a business decision; not always suitable for high-risk industries.

Why CI/CD is Important in ML

ML systems are more complex than software-only systems because they involve:

- Code (training + inference)
- Data
- Models
- Hyperparameters
- Metadata
- Environments and dependencies

CI/CD ensures these elements remain **synchronized and reproducible**.

Stages of an ML CI/CD Pipeline

1. Continuous Integration (Build Stage)

- Data scientist pushes code, model files, and metadata to repository.
- Pipeline triggers automatically.
- System builds **model artifacts** (trained model + environment + dependencies).
- Runs foundational tests:
  - Smoke tests
  - Unit tests
  - Sanity checks
- Generates **fairness, explainability, and validation reports**.

---

2. Continuous Delivery (Testing Stage)

- The artifact is deployed to a **test or staging environment**.
- Automated evaluation includes:
  - ML performance tests (accuracy, drift tests)
  - Computational performance (latency, throughput)
  - Resource checks (RAM, GPU usage)
- Human review may validate fairness, compliance, and domain-specific constraints.
- Ensures the model behaves correctly outside the development environment.

---

3. Continuous Deployment (Production Stage)

Deployment can follow multiple strategies:

- **Canary Deployment**
  - New version serves a small percentage of traffic.
  - If stable → gradually increases traffic.
- **Shadow Deployment**
  - New model runs in parallel but does NOT serve responses.
  - Useful for safety-critical testing.
- **Full Deployment**
  - Model becomes the main serving system after validation.

These strategies reduce the risk of production failures.

Key Components and Requirements of CI/CD Pipelines

A. Artifact Management

Each build must produce a consistent, versioned **artifact** containing:

- Model weights
- Source code
- Feature transformations
- Hyperparameters
- Environment specification (libraries + versions)
- Test data and evaluation results

This bundle ensures **reproducibility** and simplifies audits.

B. Automated Testing Framework

To ensure reliability, pipelines must run:

- Unit tests for functions
- Data validation tests
- Baseline comparisons
- Fairness and drift detection tests
- Edge-case tests (missing values, extreme values, corrupted data)

Fast, automated tests encourage frequent deployments and early error detection.

C. Version Control

- Git manages **code and configuration**, but not large binary model files.
- For models and datasets, tools like **DVC, MLflow, or artifact stores** are required.
- Ensures traceability for auditing and debugging.

D. Incremental CI/CD Development

- Start small with a simple working pipeline.

- Add complexity only when necessary (e.g., security checks, load tests, canary rollouts).
- Avoid building overly complex pipelines at the start.

E. Risk Mitigation through CI/CD

- CI/CD pipelines help reduce the risk of faulty models entering production.
- High-risk applications (finance, healthcare, autonomous systems) require:
  - Strong testing
  - Controlled rollouts
  - Monitoring triggers
- Canary and rollback mechanisms protect against catastrophic failures.

Tools Used

Common CI/CD orchestration tools:

- **Jenkins**, GitHub Actions, GitLab CI
- ML-specific tools: MLflow, Kubeflow, TFX, DataRobot
- Automation platforms integrate model building, testing, packaging, and deployment.

Summary (Exam-Ready)

- CI/CD pipelines automate integration, testing, packaging, and deployment of ML models.
- They ensure consistency, reproducibility, and reduce deployment risks.
- CI/CD improves collaboration, stability, and allows reliable, frequent, high-quality model releases.

---

4. Explain **deployment strategies and categories of model deployment**.
   Answer:
   Deployment Strategies and Categories of Model Deployment

   Deploying a machine learning model means taking the trained model and running it in a production environment. This requires selecting an appropriate scoring category and using

safe deployment strategies to reduce risk, downtime, and failures.

Categories of Model Deployment

Batch Scoring

• The model processes **large datasets at once**, usually through scheduled jobs (daily, hourly, weekly).
• Useful when predictions are not required immediately.
• Can be parallelized using **partitioning or sharding**, where the dataset is split into smaller parts and scored independently.
• Distributed systems like **Apache Spark** are commonly used.
• Suitable for applications like credit scoring, churn prediction, demand forecasting, etc.

Real-Time Scoring

• The model scores **one or a few records at a time**, producing an immediate result.
• Needed for interactive systems such as fraud detection, ad placement, and recommendation engines.
• Multiple model instances may run simultaneously, with a **load balancer** distributing incoming requests.
• Ensures low-latency, on-demand predictions.
• In some systems, a single record is treated as a batch of size one, creating a continuum between batch and real-time.

Deployment Strategies

Blue-Green Deployment (Red-Black Deployment)

• Purpose is to **avoid downtime** when deploying a new model version.
• Two environments are maintained:
– “Blue” = current stable version
– “Green” = new version
• The new version is deployed in parallel with the old one.
• Once validated, traffic is **switched** from blue to green.
• Allows instant **rollback** by redirecting traffic back to the blue version.
• Common in real-time scoring environments and supported by platforms like Kubernetes.

Canary Deployment (Canary Release)

• Designed to **limit risk** by releasing the new version to only a **small portion of the traffic**.
• Both old and new versions run simultaneously.
• For example, 5% of requests go to the canary model, 95% to the stable model.

• Metrics such as accuracy, latency, error rate, and unusual prediction patterns are monitored.
• If stable, the percentage is gradually increased until full rollout.
• If problems appear, the canary traffic is stopped with minimal impact.
• Supports **A/B testing**, allowing comparison of business performance between versions.

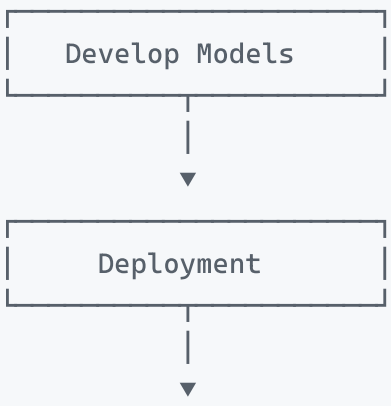
Progressive Rollouts
• New model version is released to a **small user group or region** first.
• Exposure is expanded slowly while monitoring performance and gathering human feedback.
• Reduces risk by ensuring the new model behaves correctly for different user segments.
• Similar to canary releases but focuses more on gradual, controlled user exposure.

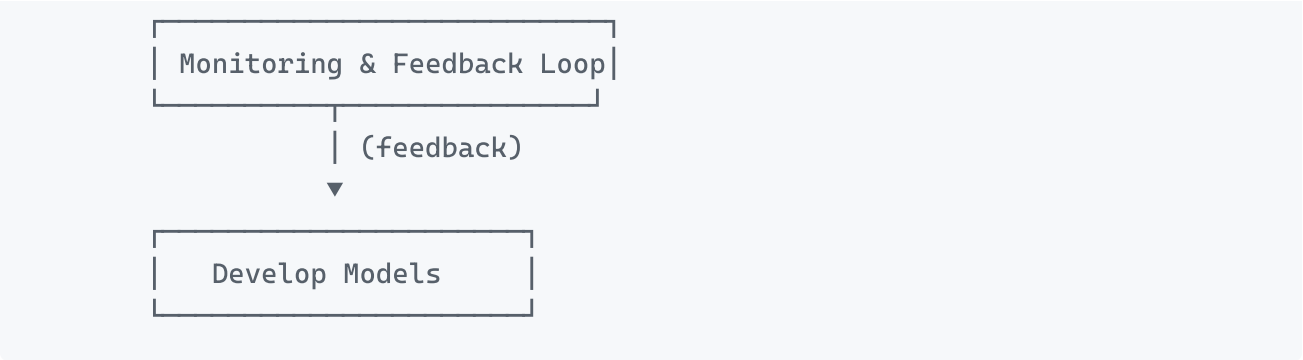
Module 5

1. With neat diagram, explain **monitoring and feedback loop highlighted in the larger context of ML lifecycle**.
   Answer:
   Monitoring and Feedback Loop in the ML Lifecycle (With Neat Diagram)

   The monitoring and feedback loop is the **final and continuous stage** of the ML lifecycle. After deployment, the model must be constantly observed for performance degradation because real-world data changes over time. This loop ensures that the model remains reliable, accurate, and aligned with business objectives by feeding information back into the development stage.

   Neat Diagram (Simple to Draw in Exam)





This shows that monitoring feeds back into development, creating a **cycle of improvement**.

Purpose of Monitoring and Feedback Loop

• Production models face changing patterns, drift, new behaviours, and unforeseen data conditions.
• Without monitoring, model quality can degrade silently and cause business damage.
• The goal is to **detect issues early**, evaluate performance, and trigger **retraining or redevelopment**.

Inputs Collected During Monitoring

• Model logs from production
• Ground truth labels (when available)
• Incoming production data
• Performance metrics and drift indicators
• Comparison against training/validation behaviour

These are analysed and sent back to development for updates.

Two Levels of Model Monitoring

Resource-Level Monitoring (DevOps)
• Checks whether the system is running as expected.
• Tracks CPU/RAM usage, network load, uptime, error rates, and throughput.
• Ensures the model server is healthy and responsive.

Performance-Level Monitoring (ML-Specific)
• Ensures the model is still accurate and relevant over time.

• Detects if model predictions drift away from real outcomes.
• Checks whether the model still matches the patterns seen during training.

Approaches to Detect Model Degradation

Ground Truth Evaluation
• Waits until actual outcomes (ground truth) are available.
• Compares current performance with training-phase metrics such as accuracy, ROC-AUC, log loss, etc.
• Business metrics (profit, risk score, conversion rate) can be monitored.
• If performance drops beyond a threshold → triggers retraining.
• Challenges: Ground truth may arrive late or be incomplete.

Input Drift Detection
• Monitors changes in incoming data even when ground truth is unavailable.
• Checks whether feature distributions (mean, SD, correlations) differ from training data.
• Causes include non-stationary data, seasonal changes, or sample-selection bias.
• Techniques include KS test, Chi-square test, or domain classifier drift detection.
• If drift is severe → retraining or model adjustments begin.

Feedback Loop Infrastructure

Logging System
• Stores model inputs, outputs, metadata, explanations, and system actions.
• Critical for debugging, auditability, and drift analysis.

Model Evaluation Store
• Tracks all model versions and their metrics over time.
• Allows comparison between deployed model and new candidates.

Online Evaluation System
• Tests new models using real production traffic.
• Methods include:
– **Champion/Challenger (Shadow Testing):** challenger model scores requests but does not serve results.
– **A/B Testing:** traffic split between two models to compare performance impacts.

Importance of the Feedback Loop

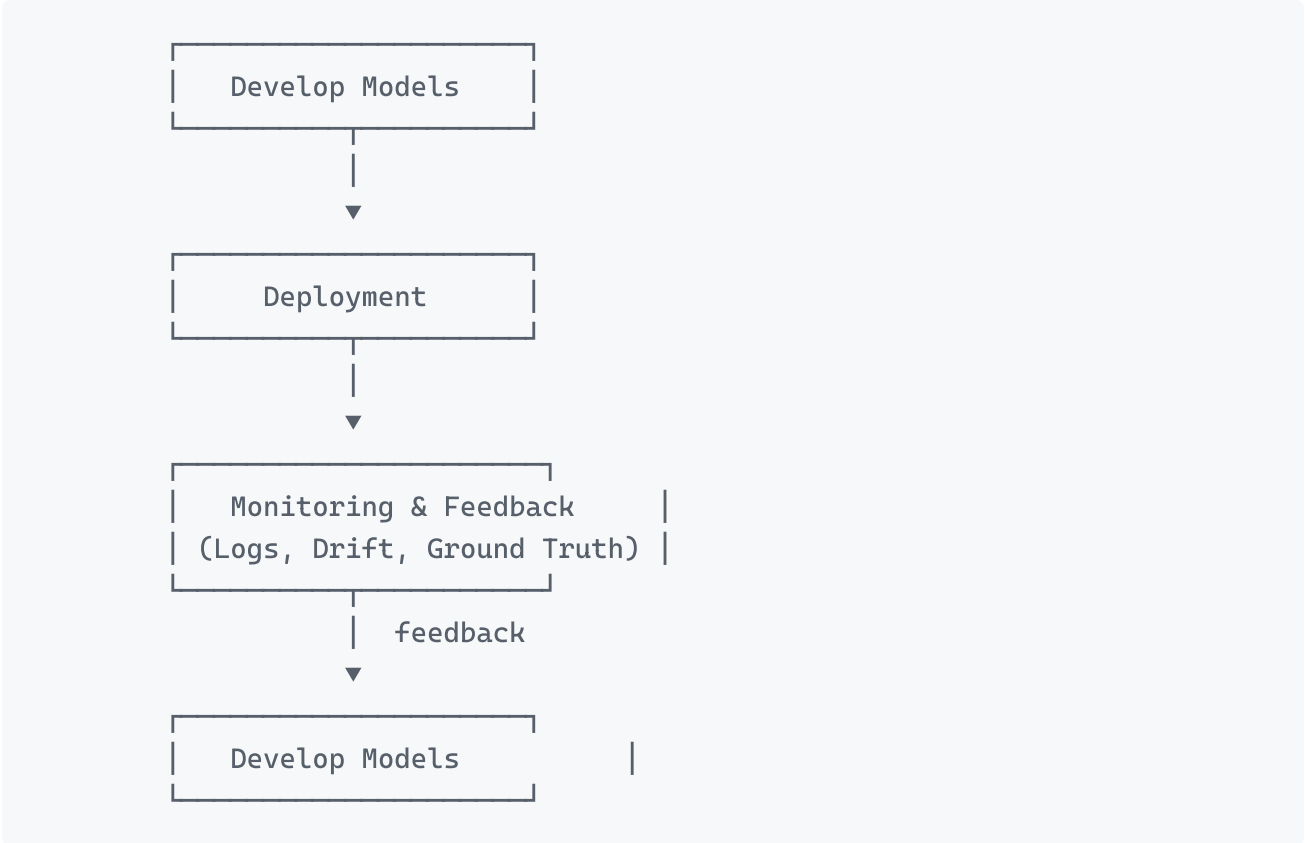
• Ensures models remain accurate, fair, and safe over long periods.
• Helps prevent silent failures.
• Supports continuous improvement, retraining, and version upgrades.
• Keeps the ML system aligned with real-world conditions.

---

2. Explain the **feedback loop** along with **logging and model evaluation**.
   Answer:
   Feedback Loop with Logging and Model Evaluation

   The feedback loop is the continuous final stage of the ML lifecycle. After a model is deployed, real-world data may change over time, causing the model's performance to degrade. The feedback loop ensures that production information flows back into the **Develop models** stage so that the model stays accurate, fair, and relevant.

   Neat Diagram (Easy to Draw in the Exam)



   This shows how monitoring continuously sends information back to model development.

The Feedback Loop Mechanism

• Once a model is deployed, the system continuously collects production information such as predictions, input data, delays, and outcomes.
• This information is analysed to check whether the model still performs as expected.
• If performance is stable → no action needed.
• If performance degrades → a retraining or redesign process is triggered.
• The goal is early detection of issues so the business is not harmed by an outdated model.
• Feedback does *not* always mean automatic retraining; instead, it alerts the data scientist to diagnose the problem.

Information feeding back into development includes:
• Model logs
• Ground truth (when available)
• Production input data
• Drift and performance metrics

Logging System

The logging system gathers and stores all events from the production model. It is the backbone of the feedback loop.

A typical log entry contains:
• **Model metadata** – model name, version, timestamp
• **Model inputs** – feature values sent for prediction (used for drift detection)
• **Model outputs** – prediction values
• **System action** – what the system did with the prediction (e.g., block a transaction)
• **Model explanation** – feature importance or Shapley values (required in regulated domains)

Logging is essential because it:
• Enables diagnosis of errors
• Supports drift detection
• Provides transparency and accountability
• Supplies historical data for retraining and audits

Model Evaluation Store

This is a centralized structure that stores evaluation data for all versions of a model. It supports two major tasks:

• **Versioning the evolution of a logical model**
– Stores all details from the training phase: dataset used, features, preprocessing steps, algorithm, hyperparameters, and evaluation metrics.
– A “logical model” represents one business problem solved by many improved versions over time.

• **Comparing performance across versions**
– All candidate models and the currently deployed model are evaluated on the **same dataset** for a fair comparison.
– The preferred dataset is newly labeled production data.
– If unavailable, the original test dataset may be reused (assuming no major drift).

After comparing candidates offline, discrepancies may still remain between offline and real-world performance. Therefore, an online evaluation setup is needed.

Online Evaluation Components

• **Champion–Challenger (Shadow Testing)**
– The active model is the “champion.”
– A new model, the “challenger,” scores the same live requests but its predictions are not used.
– Measures real-world behavior without risk.

• **A/B Testing**
– A portion of traffic goes to the new model (B), while the rest goes to the current model (A).
– Useful when the evaluation objective is tied to business metrics rather than purely prediction accuracy.
